FS1 · Full-stack para AI Engineers

TypeScript · React · Next.js · Streaming · Herramientas · Despliegue

AI Surfer · N3-AI (optativo)

Por qué este módulo

- Ofertas reales (oct. 2026): **Forward Deployed Engineer** (HappyRobot) y **AI/ML Engineer** (Galadrim) piden React/TS/Node además de Python
- El cliente no usa notebooks: usa una web
- Al terminar: una app desplegable con asistente de IA para tu portafolio

Mapa del módulo

1. TypeScript para quien viene de Python
2. React
3. Next.js: servidor vs cliente
4. Streaming
5. Herramientas + Zod + errores
6. Despliegue
7. Autenticación
8. *Discovery* con cliente

FS1.1 · De Python a TypeScript

Python	TypeScript
`list[str]`	`string[]`
`Optional[str]`	`string | undefined`
`Literal["a","b"]`	`"a" | "b"`
`async def`	`async function`

- `strict: true`, `unknown` mejor que `any`
- **Los tipos se borran al ejecutar**

FS1.1 · Unión discriminada

```
type Evento =
  | { type: "texto"; texto: string }
  | { type: "fin" };

if (e.type === "texto") e.texto; // TypeScript lo sabe
```

Es el patrón de `block.type === "text"` en la API de Claude.

FS1.2 · React en 4 ideas

- Componente = función que devuelve interfaz
- **Estado** (`useState`) -> pantalla
- Eventos: `onChange` , `onSubmit`
- Actualización funcional `setX(prev => ...)` en streaming

FS1.3 · Dos mundos

Servidor	Navegador
`process.env`, SDK, datos	`useState`, eventos
Por defecto	`"use client"`
Privado	**Público**

La clave solo vive a la izquierda. `NEXT_PUBLIC_*` = público.

FS1.3 · Arquitectura de la app

```
navegador -> POST /api/chat -> route handler
                               | Zod (400 / 422)
                               v
                          generarRespuesta() -> Claude (stream)
                               ^                    |
                               +-- consultar_pedido
navegador <- NDJSON en streaming
```

FS1.4 · Streaming en dos capas

1. Anthropic -> servidor: `client.messages.stream()` + `text_delta`
2. Servidor -> navegador: `ReadableStream` con NDJSON

- Líneas partidas -> *buffer*
- Cancelar si cierran la pestaña
- **Modo simulado**: mismo canal, sin clave

FS1.4 · Reglas de la API

- Nunca `content[0].text`: filtra `type === "text"`
- `max_tokens` holgado, `output_config: { effort }`
- **Sin** `temperature`, `top_p`, `top_k`, prefill ni `tool_choice` forzado
- Modelo: `claude-opus-5-5`

FS1.5 · Bucle de herramientas

1. Llamar y transmitir texto
2. `stop_reason === "tool_use"` -> guardar contenido **completo**
3. **Una** `tool_result` **por cada** `tool_use`, en **un** mensaje de usuario
4. Repetir con tope de vueltas

FS1.5 · Zod en las fronteras

```
const Entrada = z.object({
  id_pedido: z.string().trim().toUpperCase().regex(/^TS-\d{4}$/),
});
Entrada.safeParse(uso.input);
```

- Navegador -> servidor · Modelo -> herramienta
- Errores como datos (`is_error`), nunca secretos al navegador

FS1.5 · Tests sin red ni clave

- El bucle recibe el creador del *stream* por parámetro
- `falsoStream` emite eventos preparados
- Se prueba: orden de `tool_result`, tope de vueltas, errores
- `npm test` < 1 s

FS1.6 · Despliegue

- Repo sin `.env*` · `.env.example` documentado
- Vercel: importar -> variables de entorno -> desplegar
- Límite de gasto en la consola de Anthropic
- Revisa `maxDuration` y rota claves filtradas

FS1.7 · Autenticación

- Autenticación (quién) ≠ autorización (qué)
- Cookie de sesión `HttpOnly` · JWT · OAuth
- Se comprueba **en el servidor**
- IDOR: filtra por el usuario de la sesión
- *Rate limiting*

FS1.8 · Discovery -> demo

Problema · Usuario · Datos · Riesgos · Éxito · **Fuera de alcance**

Demo: caso feliz (60 s) -> caso de fallo -> qué falta para producción

Entregable

App adaptada a un cliente ficticio · herramienta con Zod · streaming · >= 8 tests · desplegada · documento de *discovery*

Es tu pieza de portafolio de FDE